

MisterFix — App Flutter do Cliente

Disciplina	Lab. Desenvolvimento Aplicações Móveis e Distribuídas
Sprint	Sprint 3 — Aplicativo Flutter para o Cliente
Entrega	15/06/2026
Tecnologias	Flutter 3.x · Dart 3.x · Provider · HTTP · SharedPreferences
Arquitetura	Clean Architecture (Domain / Data / Presentation)
Repositório	https://github.com/matheustitton/MrFix

1. Visão Geral do Aplicativo

O app MisterFix para o cliente é uma aplicação Flutter desenvolvida com arquitetura Clean Architecture, responsável por conectar usuários que precisam de serviços domésticos com profissionais disponíveis. O aplicativo consome a API REST implementada na Sprint 1 e utiliza polling assíncrono para refletir mudanças de estado em tempo real, atendendo ao requisito de atualização assíncrona da Sprint 3.

1.1 Telas Implementadas

Tela	Arquivo	Descrição	Requisito
Splash Screen	splash_screen.dart	Animação de entrada com verificação de sessão JWT	Não obrigatória
Login	login_screen.dart	Autenticação com e-mail/senha, toggle de modo escuro	Obrigatória
Cadastro	register_screen.dart	Registro de cliente com seleção de gênero para filtro de segurança	Obrigatória
Home (Listagem)	home_screen.dart	Listagem de pedidos, categorias de serviço e toggle de atendimento feminino	Obrigatória
Meus Pedidos	requests_screen.dart	Lista com abas: Ativas, Concluídas e canceladas	Obrigatória
Novo Pedido	new_request_screen.dart	Criação de solicitação com categoria, descrição, endereço e filtro de gênero	Obrigatória

Detalhe do Pedido	<code>request_detail_screen.dart</code>	Status em tempo real, barra de progresso animada, card do prestador e avaliação	Obrigatória
Busca de Prestadores	<code>providers_screen.dart</code>	Listagem de profissionais com filtros por gênero, categoria e ordenação	Complementar

2. Arquitetura — Clean Architecture

O aplicativo segue os princípios de Clean Architecture, organizando o código em três camadas com dependências unidirecionais: **Domain** → **Data** → **Presentation**. Cada camada tem responsabilidade única e pode ser modificada ou testada de forma independente.

2.1 Diagrama de Camadas

Camada	Pasta	Responsabilidade	Arquivos Principais
Presentation	<code>lib/presentation/</code>	Interface do usuário. Telas, widgets e providers de estado. Não conhece a fonte dos dados.	<code>screens/</code> , <code>providers/</code> , <code>widgets/</code>
Data	<code>lib/data/</code>	Implementação concreta dos repositórios. Parsing de JSON e chamadas HTTP.	<code>datasources/</code> , <code>models/</code>
Domain	<code>lib/domain/</code>	Regras de negócio puras. Entidades sem dependências externas.	<code>entities/</code>
Core	<code>lib/core/</code>	Infraestrutura compartilhada. Cliente HTTP, constantes e tema.	<code>network/</code> , <code>constants/</code>

3. Gerenciamento de Estado — Provider Pattern

O aplicativo utiliza o pacote **provider** (ChangeNotifier) para gerenciamento de estado. Três providers são registrados no topo da árvore de widgets via **MultiProvider**, garantindo que qualquer tela acesse o estado sem acoplamento direto.

Provider	Estado Gerenciado	Principais Métodos
AuthProvider	Sessão do usuário, token JWT, status de autenticação (unknown / authenticated / unauthenticated)	<code>checkAuth()</code> , <code>login()</code> , <code>register()</code> , <code>logout()</code>
ServiceRequestProvider	Lista de solicitações, solicitação selecionada, categorias, lista de prestadores, estado de polling	<code>loadRequests()</code> , <code>createRequest()</code> , <code>cancelRequest()</code> , <code>startPolling()</code> , <code>stopPolling()</code> , <code>loadProviders()</code>
ThemeProvider	Modo de tema atual (light/dark), persistido via SharedPreferences	<code>toggle()</code>

4. Integração com o Backend REST

Toda comunicação com a API é centralizada na classe **RemoteDataSource**, que utiliza o **ApiClient** para injetar automaticamente o token JWT em cada requisição autenticada. Isso garante que nenhuma tela ou provider acesse HTTP diretamente, respeitando a separação de camadas da Clean Architecture.

Método	Endpoint	Usado em	Auth
POST	<code>/auth/login</code>	LoginScreen	Não
POST	<code>/auth/register</code>	RegisterScreen	Não
GET	<code>/auth/me</code>	SplashScreen	JWT
GET	<code>/categories</code>	HomeScreen, NewRequestScreen	Não
GET	<code>/providers</code>	ProvidersScreen	Não
GET	<code>/service-requests</code>	HomeScreen, RequestsScreen	JWT
POST	<code>/service-requests</code>	NewRequestScreen	JWT
GET	<code>/service-requests/:id</code>	RequestDetailScreen	JWT
PATCH	<code>/service-requests/:id/status</code>	RequestDetailScreen	JWT
POST	<code>/ratings</code>	RequestDetailScreen	JWT

5. Atualização Assíncrona de Estado — Polling

O requisito de atualização assíncrona é implementado via **polling periódico** no **ServiceRequestProvider**. Um **Timer** é iniciado ao abrir a **HomeScreen** e executa silenciosamente a cada 5 segundos, buscando atualizações do backend sem bloquear a interface do usuário.

Fluxo do Polling:

```
startPolling() → Timer.periodic(5s) → _silentRefresh() ↓ GET  
/api/service-requests ↓ Sestatusmudou → notifyListeners() → UIatualiza automaticamente
```

O método **_silentRefresh()** não exibe indicador de carregamento, evitando interrupção visual ao usuário. Erros de rede são capturados silenciosamente, garantindo que uma falha de conexão não derrube a experiência. O polling é encerrado ao fazer logout (**stopPolling()**), prevenindo vazamentos de memória.

Critério	Polling (implementado)	WebSocket (Sprint 4)
Complexidade	Baixa — Timer nativo do Dart	Alta — requer servidor WebSocket
Latência	Até 5 segundos	Tempo real (<100ms)
Consumo de rede	Uma requisição a cada 5s	Conexão persistente
Adequação	Suficiente para Sprint 3	Ideal para Sprint 4

6. Funcionalidade de Segurança — Filtro de Gênero

O app implementa um filtro de segurança que permite à cliente solicitar exclusivamente profissionais do gênero feminino. Esta funcionalidade está presente em três pontos do fluxo:

- **HomeScreen** — toggle animado 'Apenas prestadoras femininas' que filtra a lista de prestadores exibida.
- **NewRequestScreen** — toggle 'Atendimento Feminino' que define o campo **preferred_gender: 'female'** na solicitação enviada ao backend.
- **ProvidersScreen** — filtro por gênero na busca de profissionais, com badge visual 'Profissional mulher' nos cards.

No backend, o campo **preferred_gender** na tabela **service_requests** garante que somente prestadores compatíveis recebam a notificação via MOM (Sprint 2) e vejam a solicitação disponível em seus apps.

7. Dark Mode

O aplicativo suporta alternância entre tema claro e escuro via **ThemeProvider**. A preferência do usuário é persistida no dispositivo com **SharedPreferences**, sendo restaurada automaticamente a cada abertura do app. O botão de alternância está acessível na AppBar da **HomeScreen** e na tela de Login.

Token	Light (#)	Dark (#)
-------	-----------	----------

Background	#F5F5F5	#121212
Surface	FFFFFF	#1E1E1E
Surface 2	FFFFFF	#2A2A2A
Text Primary	#1A1A1A	#F5F5F5
Text Secondary	#757575	#9E9E9E
Border	#E8E8E8	#333333
Primary	#C82828	#C82828

8. Animações e Feedback Visual

O app utiliza animações para melhorar a experiência do usuário, todas implementadas com **AnimationController** e **AnimatedContainer** nativos do Flutter, sem bibliotecas externas:

Componente	Animação	Implementação
SplashScreen	Logo com scale elástico + texto com slide e fade	<code>AnimationController + ScaleTransition + SlideTransition</code>
PrimaryButton	Scale down (0.96x) no toque	<code>GestureDetector + ScaleTransition</code>
TappableCard	Scale down (0.97x) no toque	<code>AnimationController + ScaleTransition</code>
Toggle Feminino	Deslizamento do thumb + mudança de cor do track	<code>AnimatedContainer + AnimatedAlign</code>
Category Chips	Scale down (0.93x) no toque + mudança de cor animada	<code>AnimationController + AnimatedContainer</code>
Progress Steps	Cor dos steps e conectores animada por status	<code>AnimatedContainer com duration 400ms</code>
Dark Mode Toggle	Troca de ícone e cor animada	<code>AnimatedContainer + Icon swap</code>
Login/Detail	Fade in na entrada da tela	<code>FadeTransition com CurvedAnimation</code>
NewRequest	Slide up ao abrir (bottom sheet style)	<code>SlideTransition + PageRouteBuilder</code>

9. Decisões de Design e Justificativas

Por que Provider em vez de Bloc ou Riverpod?

O pacote provider é a solução oficial recomendada pelo time Flutter para projetos de complexidade média. Sua integração com ChangeNotifier é suficiente para o escopo deste projeto e reduz a curva de aprendizado, permitindo foco na lógica de negócio e na integração com a API.

Por que polling em vez de WebSocket?

O polling com intervalo de 5 segundos atende ao requisito da Sprint 3 de atualização assíncrona sem exigir modificações no backend. A transição para WebSocket ou Firebase Cloud Messaging está planejada para a Sprint 4, conforme os hooks já presentes nos consumers do RabbitMQ (Sprint 2).

Por que Clean Architecture para um app mobile?

A separação em camadas Domain, Data e Presentation garante que o app cliente possa evoluir por 4 sprints sem refatorações destrutivas. A camada Domain permaneceu estável do início ao fim, enquanto a camada Data foi incrementada a cada sprint com novos endpoints, e a Presentation recebeu novas telas sem afetar a lógica de negócio.

10. Dependências (pubspec.yaml)

Pacote	Versão	Finalidade
flutter	SDK	Framework base
http	^1.2.1	Requisições HTTP para a API REST
provider	^6.1.2	Gerenciamento de estado
shared_preferences	^2.2.3	Persistência local do token JWT e tema
intl	^0.19.0	Formatação de datas
cupertino_icons	^1.0.6	Ícones iOS

11. Como Executar o Aplicativo

Pré-requisitos:

- Flutter SDK 3.x instalado e configurado
- Backend MisterFix rodando em **localhost:3000**
- Emulador Android ou dispositivo físico conectado

Comandos:

```
# 1. Instalar dependências flutter pub get # 2. Executar no emulador Android
flutter run # 3. Para dispositivo físico (mesma rede Wi-Fi) # Editar
lib/core/constants/app_constants.dart: # static const String baseUrl =
'http://SEU_IP_LOCAL:3000/api'; flutter run # 4. Executar no Chrome
(desenvolvimento) # Editar baseUrl para 'http://localhost:3000/api' flutter run -d chrome
```